

What Can We Deploy Today?

Checking Upgrade Batches Against Every Version Pair

[AUTHOR NAME]

[AFFILIATION]

[EMAIL]

ABSTRACT

Schema-compatibility tools check one boundary between two versions. Rolling deployments break that model in three ways: several services upgrade as a batch, old and new instances of each service run concurrently while the batch rolls, and some invariants span chains of services rather than single call edges. We state the resulting problem plainly: given a mesh of versioned, typed interfaces and a proposed batch, is every version pair that can be live on any edge during the rollout compatible? When the answer is no, the useful output is not a verdict but a plan — the subset of the batch that can ship, and the order in which to ship it. Compatibility violations translate into Horn clauses paired with ordering constraints. For the implication fragment of that system a unique largest subset exists and unit propagation finds it in linear time; conflicts — target-state failures and ordering cycles — make maximality NP-hard and are excluded conservatively. Every plan is certified by replaying its stages. Finally, we show that batch checking is still blind along the time axis: a guarantee can be weakened in a rollout where nothing depends on it, and the violation appears only when a dependent arrives several rollouts later, where a per-rollout checker necessarily blames the wrong change. A small ledger of guarantees, carried across rollouts, is enough to name the rollout that weakened the guarantee. We develop the ideas qualitatively on a nine-service mesh evolved through eleven synthetic but plausible rollouts.

needs your input

Author block above: name, affiliation, email. Also pick the title — alternatives: “Ship What You Can: Checking Microservice Upgrade Batches”; “Upgrade Batches, Version Pairs, and Rollout Order”. Delete the `nonacm` class option if the venue supplies rights info.

1 INTRODUCTION

During a rolling upgrade a mesh runs a mixture of versions. Every request may be served by an old or a new instance of its provider, and issued by an old or a new instance of its caller. A change that is compatible version-to-version can still break one of these transient pairings, and the pairing occurs only in production. This failure class is well documented: a study of 123 upgrade failures across eight widely used distributed systems attributes about two thirds to incompatible interaction between two coexisting versions, 40% of those through network messages [19].

Existing tools check pairs of versions at single boundaries. Pact verifies consumer-provider contracts and gates deployment on recorded

verifications [15]; Buf classifies protobuf diffs by wire compatibility [4]; schema registries enforce backward/forward compatibility per topic [5]; `oasdiff` classifies OpenAPI diffs [18]. These are sound answers to a smaller question. Three things are missing:

- (1) **Batches.** Per-boundary verdicts do not compose into a verdict for a set of simultaneous upgrades, and no per-boundary tool answers the operative question when a batch fails: which subset can ship?
- (2) **Order.** Many “breaking” changes are only *order*-constrained. A provider that starts requiring a field its callers have agreed to send is safe exactly when every caller finishes rolling before the provider starts. The correct output is a schedule, and pairwise tools cannot express one.
- (3) **History.** A guarantee can be weakened in a rollout where nothing depends on it; every check passes and the change ships. When a dependent appears several rollouts later, the check that finally fails sits in the wrong diff. Attribution requires state that survives across rollouts.

Problem statement

Given a mesh of services with versioned, typed interfaces, a current deployment state, and a proposed upgrade batch: (1) decide whether every version pair that can be live on any edge during the rollout is compatible, and whether declared cross-service dataflow chains hold in the resulting state; (2) if not, produce a subset of the batch and a stage order that pass the same checks, certified by replaying the stages; (3) when a chain violation is exposed by a new dependent, identify the earlier rollout that weakened the guarantee.

We built a proof-of-concept checker, GUS (*global upgrade safety*), over OpenAPI interfaces, and evolved a nine-service mesh through eleven synthetic rollouts to exercise it. The contribution of this paper is the problem formulation and the shape of the solution: compatibility as a check over all live version pairs (§2–3), plans as a Horn system with ordering constraints and a replay certificate (§4), and a guarantee ledger for attribution across rollouts (§5). All evaluation is qualitative (§6); §8 states what the approach cannot see.

needs your input

One paragraph only you can write: who runs a reviewed batch in practice (platform teams shipping umbrella releases, monorepo deploy trains, on-prem vendors), and why that is your target user. Teams that deploy each service independently and continuously have no batch, so this framing matters for the paper’s relevance claim.

2 MODEL

A mesh is a directed graph $G = (S, E)$; an edge $e = (u \rightarrow v)$ is an RPC endpoint on provider v called by u . Each service has totally ordered versions. A *deployment state* is a map $\theta : S \rightarrow \mathbb{V}$; an upgrade batch is a partial map U , and the target state is $\theta' = \theta \oplus U$.

Each version of a service fixes four schemas per edge it participates in: the caller's request body (Send) and response expectation (Expect), and the provider's request schema (Accept) and response body (Return). Callers and providers are typed separately because caller code drifts from provider contracts. When a caller declares no outbound contract, the checker anchors $\text{Send} := \text{Accept}(e, \theta)$ and $\text{Expect} := \text{Return}(e, \theta)$ at the *old* provider version — the caller is assumed to have been built against the contract that was live — which reduces the check to ordinary backward-compatibility rather than inventing caller-side change.

Rolling semantics: replacing instances service by service means that, at any instant, each upgrading service has live instances at $\theta(s)$, at $\theta'(s)$, or both. We assume at most two versions of a service are ever live (no canary with three), and that the mesh routes a request to any live provider instance. Consequently the version pairs that can be live on edge $u \rightarrow v$ during an unconstrained rollout of U are exactly $\{\theta(u), \theta'(u)\} \times \{\theta(v), \theta'(v)\}$. A *staged* rollout restricts which pairs occur: if u finishes before v starts, the pair $(\theta(u), \theta'(v))$ never exists.

3 EDGE COMPATIBILITY OVER VERSION PAIRS

Compatibility of one version pair is subtyping with the standard variance split, as in binary session subtyping [8]. Writing i for a caller version and j for a provider version on edge e :

$$\text{ok}_e(i, j) \triangleq \text{Send}_e(i) \leq_{\text{req}} \text{Accept}_e(j) \wedge \text{Return}_e(j) \leq_{\text{res}} \text{Expect}_e(i).$$

The request relation $\leq_{\text{req}}$ requires everything the caller may send to be admitted by the provider; the response relation $\leq_{\text{res}}$ requires everything the provider may return to be understood by the caller. Both are structural over a JSON type language: primitives under a partial order, enums as value sets, records with width and optionality rules, unions with existential matching, and equi-recursive references handled by cycle detection with a coinductive assumption at back-edges. The primitive order is strict: an integer is *not* assumed readable where a string is declared, because production decoders reject such tokens. Findings that are range risks rather than shape breaks (e.g. a response widening from 32- to 64-bit integers) are reported as warnings and do not fail a batch.

Edge compatibility for the transition $\theta \rightarrow \theta'$ is the conjunction over all live pairs:

$$\text{EdgeOK}(e, \theta, \theta') \triangleq \bigwedge_{\substack{i \in \{\theta(u), \theta'(u)\} \\ j \in \{\theta(v), \theta'(v)\}}} \text{ok}_e(i, j).$$

The (θ, θ) conjunct is the precondition that the current state is internally consistent, checked separately; the two mixed conjuncts and the (θ', θ') conjunct are what the transition adds. $\text{GUS}(\theta, U)$ is EdgeOK over all edges touched by U , together with chain integrity in the target state (§5); a chain already broken in the baseline is reported as pre-existing rather than attributed to the batch.

PROPOSITION 3.1 (ADEQUACY FOR EDGES). *Assume at most two live versions per service and uniform routing, and that θ is consistent. The batch U , rolled without ordering constraints, exposes no incompatible (caller, provider) version pair on any edge iff $\text{EdgeOK}(e, \theta, \theta')$ holds for every edge e .*

PROOF SKETCH. Each direction is immediate from the pair enumeration above: every reachable pair is one of the four conjuncts, and each mixed or target pair is realized by some interleaving (roll only v partway; only u ; complete both). $\square$

We scope the proposition deliberately: it speaks of edge-local pairs only. Chain integrity (§5) is checked in the endpoint states θ and θ' , not in transient mixtures; a chain that breaks only mid-rollout is outside the current formulation, and we flag this honestly in §8.

The value of checking mixed pairs is that the two relations are asymmetric, so the same change can break one pair and leave the other intact. In the case study, a provider adds a value to a response enum while its caller upgrades in the same batch. The pair (old caller, new provider) fails — an old caller can receive the new value and has no case for it — while the pair (new caller, old provider) passes, since old provider instances return only the original values. The violation is therefore not “this change is breaking” but “these two versions must not coexist,” which is an ordering fact; the next section turns such facts into plans.

4 FROM VIOLATIONS TO PLANS

When $\text{GUS}(\theta, U)$ fails, the operative question is what subset of U can ship, and how. Define a *plan* as a subset $U' \subseteq U$ together with a partition of U' into ordered stages.

Definition 4.1 (Plan safety). A plan is safe from θ iff, taking $\theta_0 = \theta$ and $\theta_k = \theta_{k-1} \oplus U'_k$ for stage k , each stage satisfies $\text{GUS}(\theta_{k-1}, U'_k)$.

By Proposition 3.1 applied stage by stage, a safe plan exposes no incompatible edge pair at any point of its execution, and plan safety is directly checkable: *replay* the stages, running the full check against each accumulated intermediate state. The checker always emits this certificate check; the solver below is a candidate generator whose output is verified by replay, so its soundness does not rest on the completeness of any encoding.

Generating candidates. Each violated conjunct pins one service of the batch at its new version and one at its old version, and therefore reads as an ordering fact. If the pair (old u , new v) is incompatible on either leg, then new- v instances must never coexist with old- u instances: if u 's contract cannot change ($u \notin U$, or u has no declared outbound contract), the violation is unconditional and v is excluded ($\neg x_v$); otherwise v may ship only if u ships first — the definite clause $\neg x_v \vee x_u$ together with the precedence $u < v$. Incompatibility of the symmetric pair (new u , old v) swaps the roles. Incompatibility of the target pair (θ', θ') pins both services at their new versions; no order avoids that pair, so the two upgrades conflict outright.

PROPOSITION 4.2 (EASY FRAGMENT). *For clause sets containing only definite clauses ($\neg a \vee b$) and negative units ($\neg a$), the true-sets of*

satisfying assignments are closed under union; hence a unique maximum exists, and propagating falsity from the units computes it in $O(|\text{clauses}| + |U|)$.

PROOF SKETCH. If $a \in T_1 \cup T_2$ then $a \in T_i$ for some i , so any clause $a \rightarrow b$ puts $b \in T_i \subseteq T_1 \cup T_2$; negative units hold in both. The union of all models is then the maximum model. Propagation is the classical linear-time procedure for Horn theories [7]. $\square$

PROPOSITION 4.3 (HARD BOUNDARY). *Once conflict clauses $\neg x_u \vee \neg x_v$ are admitted, deciding a maximum-cardinality satisfying subset is NP-hard.*

PROOF SKETCH. Such clauses are still Horn, so satisfiability stays linear; but union-closure fails ($\{u\}$ and $\{v\}$ are models, $\{u, v\}$ is not), and maximizing true variables with one conflict clause per graph edge is maximum independent set. This is the Max-Ones classification for this constraint family [12]. $\square$

The two propositions dictate the design. Unit propagation resolves the easy fragment exactly. Conflicts — target-pair failures, and cycles in the precedence relation — are resolved conservatively: the checker excludes every member of a conflicting group and re-propagates, rather than making an arbitrary choice or solving an NP-hard optimization inside a safety tool. Cycles arise naturally. In the case study, checkout starts requiring an idempotency key its old callers do not send, so frontend must finish rolling first; in the same version it changes a response field from string to integer while frontend upgrades to expect the integer, so checkout must finish first — old checkout instances still answer with strings. The two constraints are contradictory: no rolling order ships the pair, and the checker excludes both. The result is sound but not maximum-cardinality, and the tool says so. Surviving precedences are topologically sorted into the plan’s stages, and the plan is certified by replay (Definition 4.1).

5 CHAINS AND THE GUARANTEE LEDGER

Some invariants span more than one edge. In the case-study mesh, checkout mints an order identifier that must arrive intact at the email service; the intermediate schemas mark the field optional, which is precisely why no edge-local check fires if checkout stops guaranteeing it. The failure class is real: enforcing required fields at intermediate hops of forwarding chains caused enough production incidents that protobuf removed required entirely [9].

The checker supports declared chains: a source annotates the field that originates an identity (x -provides), a sink annotates where it must arrive (x -requires), and intermediates annotate only renames (x -alias). $\text{chain_ok}(K, \theta)$ enumerates the call paths from source to sink and requires the identity to be present, non-null, and type-stable at the source and at every intermediate hop, validated against what each hop *sends onward* (its outbound contract) — which is what makes a rename detectable at all. Because a mesh may route the identity along several paths, every simple path between source and sink is checked. Identity types must be stable end-to-end: an identity is stored and joined on, not merely parsed.

Why one batch is not enough. Chain integrity at a state θ_k is a predicate on that state alone, but the change responsible for a violation may sit in an earlier rollout. The concrete sequence from the case study: rollout i makes checkout’s `shipment_ref` optional while no service requires it — there is no chain, nothing fails, the change ships; rollout $k > i$ adds the requirement in email; the check at k fails. The only change a per-rollout checker can act on at step k is email’s, and excluding email is what the solver correctly does for that batch — but the defect was introduced at step i , and the repair belongs at checkout.

The fix is bookkeeping, not new theory. The checker maintains a *ledger*: after each rollout, for every provided identity — whether or not anything requires it yet — it records the guarantee as a tuple (provider service, field, type, required, nullable, carrying paths), and appends an event when the tuple changes: born, eroded (required became optional, or non-null became nullable), restored, demanded, violated. The ledger is a JSON file persisted between invocations; steps already recorded are skipped, so it accumulates across rollouts exactly like the mesh it describes. When a chain check fails, the ledger reports the most recent weakening event for that identity alongside the failure. Attribution within the failing batch is by reverting each on-path upgrade in isolation and re-checking; a change whose revert repairs (or dissolves) the chain is marked a culprit. This revert-and-recheck style of attribution is the static analogue of the counterfactual reasoning used by lineage-driven fault injection [2], and we claim no more for it than that.

For the sequence above the ledger output is (abridged):

```
identity "shipment-tracking" -- EXPOSED
  born    @03: checkout provides shipment_ref
           (string, required)
  eroded  @07: required -> optional at checkout
  demanded @11: required by email -- REJECTED
  violated @11: chain-weakened
           -- guarantee last weakened at step 07
```

Step 11’s batch is correctly blocked; step 07 is correctly named. Neither statement is available to a checker that sees one rollout at a time.

6 CASE STUDY

We ported Google’s Online Boutique demo to a nine-service mesh (frontend, checkout, shipping, product catalog, currency, cart, recommendation, payment, email; 14 RPC edges) with OpenAPI interfaces, including caller-declared outbound contracts, and wrote an *evolution suite*: eleven rollout steps shaped like ordinary feature work, where each step’s baseline is what the previous step’s plan actually shipped. The scenarios are synthetic and constructed to jointly cover the checker’s rules; the suite doubles as its regression oracle (expected verdicts, exact safe subsets, stage orders, and chain outcomes are asserted). Table 1 summarizes; three steps carry the argument.

Order (step 03). Shipping begins requiring an `account_tier` field that both callers simultaneously upgrade to send. Every pairwise tool classifies the provider change as breaking. The checker instead emits a two-stage plan — callers, then shipping — and the replay certificate passes: the only bad pair (old caller, new provider) never exists under that order.

Target state (step 10). Shipping accepts an optional enum-valued promo code; frontend upgrades to always send a free-form string. Both mixed pairs pass: old frontends omit the optional field, and old shippings ignore

Table 1: The eleven-step evolution suite. Bold rows are outcomes a per-boundary, per-rollout checker cannot produce.

Step	Outcome	Notes
01 wishlist	ships	additive response fields
02 cart	blocked	response guarantee regressed
03 accounts	ships in 2 stages	callers first, then shipping
04 payments	blocked	response enum widened; nullable id
05 payments	ships in 2 stages	checkout before payment
06 money, expand	partial	widening ships as a warning; a map value change is blocked
07 money, contract	ships in 2 stages	four violation kinds, one order; quietly weakens a guarantee
08 tracing	partial	rename without alias: only the chain fails
09 tracing	ships	alias bridges the rename
10 promo	blocked	every mixed pair passes; only the target pair conflicts
11 notifications	blocked	step-07 weakening exposed; ledger names step 07

the unknown field. Only the (θ', θ') pair conflicts. Registry-style backward/forward checking relates successive versions of *one* schema; this failure lies between two parties' new schemas, a pair no per-schema check ever relates — here it is the entire failure.

History (steps 07 and 11). As in §5: the weakening ships silently at step 07 because nothing requires the identity; the requirement arrives at step 11 and is blocked; the ledger names step 07. Without the ledger, step 11's failure is attributed to email, which is the one service that did nothing wrong.

7 RELATED WORK

Per-boundary compatibility checking is mature. Pact's broker verifies a candidate against the versions recorded as deployed — a dynamic, example-based check of mixed pairs for one consumer-provider relationship [15]. Confluent's `FULL_TRANSITIVE` mode is the both-directions condition for pub/sub topics [5]. Buf [4] and `oasdiff` [18] classify two-version diffs with direction awareness. None of these composes verdicts across a mesh, outputs an order, computes a shippable subset, or spans multiple hops. Mixed-version operation has long been studied at the node level: Ajmani et al. run simulation objects so nodes interoperate across versions during asynchronous upgrades [1]; Service Weaver avoids cross-version communication entirely by routing each request within one application version [10]; Amazon's rollback-safety guidance prescribes two-phase serialization changes as an operational discipline [16]. Typed evolution has been addressed with language support (contract evolution with runtime adaptation [6]) and with bidirectional lenses that translate between schema versions [14] — repair, where this work does detection. The per-pair subtype relations are those of binary session subtyping [8]; we do not use multiparty session types [11], since no global protocol is checked here. The solver core is Dowling–Gallier propagation [7], and the hardness boundary is the Max-Ones classification [12]. That declared versions understate breakage is well evidenced [13, 17].

8 LIMITATIONS

Specs, not behavior. The checker judges declared contracts. Measured drift between production APIs and their OpenAPI documents is substantial [3], so a passing verdict means “no statically visible hazard in the declared mesh”; failing verdicts are near-certain wire breaks. The asymmetry should

be stated wherever the tool is used. *Chains in transient states.* Chain integrity is checked in the endpoint states of each rollout, not in mid-rollout mixtures (Proposition 3.1 deliberately excludes it); extending the pair-enumeration argument to chains is open. *Two live versions.* Canary deployments hold three or more versions; the pair enumeration generalizes, the tool does not yet. *Annotations.* Chains require hand-written provides/requires/alias annotations and caller-declared outbound contracts; stale annotations corrupt verdicts silently. Inferring both from traffic or source is the obvious next layer of this work. *Conservative choices.* Conflict groups and culprit sets are excluded wholesale where choosing among them is NP-hard; a weighted formulation would let operators express preferences. *Response-carried identities.* Chain discovery follows forward call edges, so identities carried in responses are out of scope. *Rollback.* An aborted rollout traverses the same pairs in reverse; the model covers it by swapping θ and θ' , the tool does not yet.

needs your input

Evaluation plan: commit to what you will actually do next (replaying public OpenAPI version histories; reproducing documented upgrade failures from the SOSP'21 taxonomy as mesh scenarios; or a case study with a platform team's umbrella releases), and what you will measure.

needs your input

Acknowledgments, venue footnote, and whether to link the artifact repository.

REFERENCES

- [1] Sameer Ajmani, Barbara Liskov, and Liuba Shrira. 2006. Modular Software Upgrades for Distributed Systems. In *Proc. ECOOP*.
- [2] Peter Alvaro, Joshua Rosen, and Joseph M. Hellerstein. 2015. Lineage-driven Fault Injection. In *Proc. SIGMOD*.
- [3] APiContext. 2026. The APiDrift White Paper. <https://apicontext.com/resources/api-drift-white-paper/>.
- [4] Buf Technologies. 2026. Buf breaking change detection rules. <https://buf.build/docs/breaking/rules/>.
- [5] Confluent, Inc. 2026. Schema Evolution and Compatibility (Confluent Schema Registry). <https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html>.
- [6] João Costa Seco, Paulo Ferreira, Hugo Lourenço, Carla Ferreira, and Luís Ferrão. 2020. Robust Contract Evolution in a TypeSafe MicroServices Architecture. In *The Art, Science, and Engineering of Programming*.
- [7] William F. Dowling and Jean H. Gallier. 1984. Linear-time algorithms for testing the satisfiability of propositional Horn formulae. *Journal of Logic Programming* 1, 3 (1984), 267–284.
- [8] Simon Gay and Malcolm Hole. 2005. Subtyping for session types in the pi calculus. *Acta Informatica* 42, 2–3 (2005), 191–225.
- [9] Google (Protocol Buffers team). 2026. Protocol Buffers Dos and Don'ts. <https://protobuf.dev/best-practices/dos-donts/>.
- [10] Google (Service Weaver team). 2026. Service Weaver documentation: versioned rollouts. <https://serviceweaver.dev/docs.html>.
- [11] Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2008. Multiparty Asynchronous Session Types. In *Proc. POPL*.
- [12] Sanjeev Khanna, Madhu Sudan, Luca Trevisan, and David P. Williamson. 2001. The Approximability of Constraint Satisfaction Problems. *SIAM J. Comput.* 30, 6 (2001), 1863–1920.
- [13] Alexander Lercher, Johann Glock, Christian Macho, and Martin Pinzger. 2024. Microservice API Evolution in Practice: A Study on Strategies and Challenges. *Journal of Systems and Software* 215 (2024).
- [14] Geoffrey Litt, Peter van Hardenberg, and Orion Henry Good. 2020. Project Cambria: Translate your data with lenses. *Ink & Switch technical report* (2020). <https://www.inkandswitch.com/cambria/>.
- [15] Pact Foundation. 2026. Can I Deploy (Pact Broker documentation). https://docs.pact.io/pact_broker/can_i_deploy.

- [16] Sandeep Pokkunuri. 2020. Ensuring rollback safety during deployments. Amazon Builders' Library. <https://aws.amazon.com/builders-library/ensuring-rollback-safety-during-deployments/>.
- [17] Steven Raemaekers, Arie van Deursen, and Joost Visser. 2014. Semantic Versioning versus Breaking Changes: A Study of the Maven Repository. In *Proc. SCAM*.
- [18] Tufin (oasdiff maintainers). 2026. oasdiff: OpenAPI diff and breaking changes. <https://github.com/oasdiff/oasdiff>.
- [19] Yongle Zhang, Junwen Yang, Zhuqi Jin, Utsav Sethi, Kirk Rodrigues, Shan Lu, and Ding Yuan. 2021. Understanding and Detecting Software Upgrade Failures in Distributed Systems. In *Proc. SOSP*.